

AI-Based Document Classification System

Technical Documentation

Item	Value
Project	AI-Based Document Classification System
Prepared by	M Shayan Khan
Internship	SPS Summer Internship 2026
Domain	Government records retention — Library of Virginia (LVA) General Schedules
Backend	Node.js + Express 5
Frontend	HTML5 + CSS3 + Vanilla JavaScript
Vector Database	Zilliz Cloud (Milvus)
Embeddings + LLM	Cloudflare Workers AI
Object Storage	Cloudflare R2
Document Version	1.0
Status	Feature-complete; deployment optional

Table of Contents

- 1. [Introduction](#)
- 2. [Project Overview](#)
- 3. [Background Concepts](#)
- 4. [System Architecture](#)
- 5. [Technology Stack](#)
- 6. [End-to-End Workflow](#)
- 7. [Techniques Applied](#)
- 8. [Internal REST API Reference](#)
- 9. [External APIs and Services](#)
- 10. [Data Model](#)
- 11. [Configuration](#)
- 12. [Security](#)

- 13. [Error Handling Strategy](#)
- 14. [Diagnostic and Maintenance Tooling](#)
- 15. [Frontend](#)
- 16. [Setup and Local Development](#)
- 17. [Deployment](#)
- 18. [Future Enhancements](#)
- 19. [Appendix A: Glossary](#)
- 20. [Appendix B: File Reference Map](#)

1. Introduction

1.1 Purpose

This is the consolidated technical reference for the AI-Based Document Classification System: architecture, processing pipeline, techniques applied, internal/external APIs, data model, and operational tooling — written for technical reviewers and anyone who will maintain or extend the project.

1.2 Related Documents

File	Contents
1.PROJECT_SPECIFICATION.md	Original requirements
2.ARCHITECTURE.md	Layered-architecture principles
3.AI_RULES.md	Coding standards
4.TECHNICAL_DOCUMENTATION.md	This document — the as-built reference
context-capsule.md / workflow.md	Narrative context and workflow walkthrough
timeline.md / how-to-live.md	Internship plan and deployment guide

Where this document and the earlier specification disagree, **this document describes the system as actually built** and takes precedence.

1.3 Scope Note: Specification vs. Implementation

The delivered system deliberately extends the original specification:

Area	Original Specification	As Built
Target schedule	GS-17 only	GS-17 (88) + GS-02 (20) + GS-14 (11), schedule-agnostic engine
OCR	Excluded	Selective per-page OCR with a persistent worker pool

Area	Original Specification	As Built
AI provider	OpenAI API	Cloudflare Workers AI (embeddings + LLM)
Object storage	Local uploads / only	Cloudflare R2 (local disk not used)
Reranking	Not specified	Two-stage: metadata keyword boost + IDF confusable-cluster amplification
Confidence	Not specified	Composite score with linguistic certainty analysis
Hardening	Not specified	CORS lockdown, rate limiting, endpoint gating, CVE patching

2. Project Overview

2.1 The Problem

State, county, and municipal agencies generate large volumes of records daily — case files, arrest records, dispatch logs, crash reports, impound slips, EMS records, and more. Every record is governed by the **Library of Virginia (LVA) General Records Retention Schedules**: each category (a **series**) has a mandatory **retention period** and **disposition method**.

Manual classification is slow, inconsistent, and doesn't scale. Keyword search fails because real documents use varied phrasing, contain OCR noise, and hinge on subtle legal distinctions (e.g. *resolved* vs. *unresolved*) that change the retention rule entirely.

2.2 The Solution

A full-stack app that accepts an uploaded document and returns its LVA classification with reasoning, built on a **hybrid retrieval pipeline**:

1. **Semantic vector search** finds the series closest in meaning, not keywords.
2. **Deterministic reranking** separates near-identical series using metadata signals.
3. **A similarity gate** rejects documents that don't belong to any known schedule.
4. **RAG** presents top candidates and document text to an LLM, which picks one series and explains why.
5. **Composite confidence scoring** combines similarity with the model's linguistic certainty.

2.3 What the System Returns

Field	Source
Schedule / Series Number & Title	Milvus / local schedule data (series validated against LLM pick)
Series Description, Retention, Disposition	Milvus / local schedule data

Field	Source
AI Reasoning	LLM output
Confidence	Computed (similarity + certainty boost)
Top 3 Candidates	Milvus search, post-rerank
Out-of-Scope flag	Similarity gate

All statutory metadata originates from the retrieval layer — the LLM never invents metadata, it only selects a series number and writes an explanation.

3. Background Concepts

3.1 LVA General Schedules

A **General Schedule (GS)** is a published set of retention rules covering one functional area of government:

Schedule	Title	Series in dataset
GS-17	Law Enforcement, Fire and Emergency Services	88
GS-02	Administrative Records	20
GS-14	Personnel Records	11

3.2 Record Series

A **series** is one record category within a schedule. Besides statutory fields (number, title, description, retention, disposition), each series here carries enrichment fields added by this project: **text_to_embed** (semantic summary used for vectorisation), **scope_notes** (what it covers / excludes), **key_criteria** (discriminating keywords/phrases), and **exclusions** (series it's commonly confused with).

3.3 Retention Period and Disposition Method

Retention period is how long a record must be kept, often relative to an event ("**3 Years after...no longer in use**", "**100 Years after birth**"). **Disposition method** is the legally mandated end state: *Confidential Destruction*, *Non-confidential Destruction*, *Permanent In Agency*, or *Archives*.

3.4 Vector Embedding

An **embedding** maps text to a fixed-length vector (here, 768 numbers) so texts with similar meaning sit close together, measured by **cosine similarity** (0 = unrelated, 1 = identical direction).

3.5 Retrieval-Augmented Generation (RAG)

Rather than classifying from its own memory, the system first **retrieves** the most relevant reference material (top candidate series with scope notes) and places it directly in the prompt, so the model reasons **only over the supplied candidates** — grounding the output and constraining the answer space.

4. System Architecture

4.1 Layered Architecture

The backend enforces strict, unidirectional layering — no layer skips another.

```
Browser (HTML / CSS / vanilla JS)
| HTTP (multipart/form-data, JSON)
v
Routes      routes/*.js      - endpoint definitions only
v
Middleware   middleware/*.js   - rate limiting, upload parsing, error handling
v
Controllers  controllers/*.js  - request orchestration, response assembly
v
Services     services/*.js     - all business logic and external calls
v
External systems  Cloudflare R2 · Zilliz/Milvus · Cloudflare Workers AI · filesystem
```

Routes/controllers never call Milvus or Cloudflare AI directly; services never touch `req/res`; utilities know nothing about Express; no circular dependencies.

4.2 Request Lifecycle

```
Browser -> Route -> Rate-limit middleware -> Upload middleware (Multer)
      -> Controller (orchestration)
      -> uploadService -> r2Service      (archive to R2)
      -> extractionService -> ocrService (extract text)
      -> textCleaningService      (normalise text)
      -> milvusService -> embeddingService (embed + vector search)
      -> rerankingService      (rerank candidates)
      -> classificationService      (RAG classification)
      -> Controller assembles JSON
      -> Error middleware (only on failure)
Browser <- JSON response
```

4.3 Directory Structure

```
DOC-CLASS-CC/
+-- 1.PROJECT_SPECIFICATION.md · 2.ARCHITECTURE.md · 3.AI_RULES.md
+-- 4.technical_documentation.md <- this document
+-- context-capsule.md · workflow.md · timeline.md · how-to-live.md
|
+-- backend/
  +-- server.js      Express entry point, middleware mounting
  +-- package.json   Dependencies, scripts, engine pin (Node ≥ 22.13)
  +-- .env / .env.example  Secrets (git-ignored) / template
  |
  +-- config/        app.config.js · constants.js
  +-- middleware/     uploadMiddleware · rateLimitMiddleware · errorMiddleware
  +-- routes/         index · uploadRoutes · extractionRoutes (dev-only, gated)
  +-- controllers/    uploadController (orchestrator) · extractionController
  |
  +-- services/
    +-- uploadService.js  R2 upload + document metadata assembly
    +-- r2Service.js      Cloudflare R2 (S3 SDK) put / get
    +-- extractionService.js  PDF / DOCX / XLSX routing + fallback chain
    +-- ocrService.js     Persistent Tesseract worker pool, selective OCR
    +-- textCleaningService.js  7-stage cleaning + context-window builder
    +-- embeddingService.js  Cloudflare BGE embeddings (768-dim)
    +-- milvusService.js   Zilliz/Milvus connection, schema, search, lookup
    +-- rerankingService.js  Stage-2 IDF confusable-cluster reranker
    +-- classificationService.js  RAG prompt, LLM call, confidence scoring
  |
  |
```

```
+-- utils/logger.js · data/schedules/ (gs-17/02/14.json)
+-- views/index.html · public/css,js/ (UI, theme, upload)
+-- scripts/                               Diagnostic / benchmark / ingestion CLIs (§14)
```

5. Technology Stack

5.1 Runtime and Framework

Component	Version	Role
Node.js	≥ 22.13	Async runtime; floor required by patched <code>pdfjs-dist</code> .
Express.js	^5.2.1	HTTP routing and middleware chaining.

5.2 Upload and Extraction

Library	Version	Role
<code>multer</code>	^2.2.0	Multipart parsing into an in-memory buffer — nothing hits local disk.
<code>pdf-parse</code>	^1.1.4	Primary PDF text extraction.
<code>pdfjs-dist</code>	^6.1.200	Fallback for malformed PDFs; also rasterises pages for OCR.
<code>mammoth</code>	^1.12.0	DOCX raw-text extraction.
<code>xlsx</code> (SheetJS)	^0.18.5	XLSX / XLS sheet-to-CSV conversion.
<code>tesseract.js</code>	^7.0.0	OCR engine for scanned / image-only PDFs.
<code>canvas</code> / <code>@napi-rs/canvas</code>	^3.2.3	Rasterises PDF pages to PNG at 2× scale before OCR.

5.3 AI, Vectors, and Storage

Component	Identifier	Role
Embedding model	<code>@cf/baai/bge-base-en-v1.5</code>	768-dim dense embeddings (Cloudflare Workers AI).
LLM	<code>@cf/meta/llama-3.1-8b-instruct</code>	RAG candidate selection and reasoning, temperature 0.1.

Component	Identifier	Role
Vector DB	@zilliz/milvus2-sdk-node ^3.0.4	Collection <code>gs17_records</code> , <code>FloatVector(768)</code> , <code>AUTOINDEX</code> , <code>COSINE</code> .
Object storage	@aws-sdk/client-s3 ^3.1081.0	S3-compatible archival buffer for uploads (Cloudflare R2).

5.4 Cross-Cutting

Library	Version	Role
dotenv	^17.4.2	Environment variable loading.
cors	^2.8.6	Default-deny unless <code>CORS_ORIGIN</code> is set.
morgan	^1.11.0	HTTP request logging (<code>dev</code> format).
express-rate-limit	^8.7.0	Per-IP throttling on cost-sensitive endpoints.
nodemon	^3.1.14 (dev)	Auto-restart during development.

6. End-to-End Workflow

The `POST /api/v1/upload` request runs the following pipeline (stage numbers per `controllers/uploadController.js`):

```

[1] Multer parses the multipart body into an in-memory buffer.
    MIME type checked against the whitelist; > 2 MB rejected with HTTP 400.
    |
    v
[2] Archive + extract run CONCURRENTLY (Promise.all):
    +- uploadService.processUpload -> r2Service.uploadToR2 (archive copy to R2)
    +- extractionService.extractUploadedFile(buffer, name) (text from the buffer)
      | PDF: pdf-parse -> pdfjs-dist fallback -> OCR fallback
      | DOCX: mammoth
      | XLSX: SheetJS (each sheet -> CSV block)
      Extraction failure degrades softly; R2 failure fails the whole request.
    |
    v
[2b] textCleaningService.cleanExtractedText – 7-stage normalisation (§7.3).
    |
    v
[3] Vector search (only if cleaned text exists):
    embeddingService.generateEmbedding -> 768-dim query vector
    milvusService.searchSimilarRecords -> top 8 candidates (cosine, nprobe 32)
    Each candidate enriched with scope_notes / key_criteria / exclusions
    from the local schedule JSON (keyed by series_number).
    |
    v
[3a] Stage-1 rerank – proportional key_criteria keyword boost, ceiling +0.030.
    |
    v

```

```

[3b] Stage-2 rerank – IDF-weighted boost with amplified ceiling for
      mutually-excluding "confusable clusters" (§7.8). Produces
      top_3_candidates and the top-5 classification set.
      |
      v
[4] Similarity gate: top candidate score < 0.65 -> out_of_scope = true,
      LLM skipped, top 3 returned "for reference only". Otherwise continue.
      |
      v
[4a] RAG classification – builds the head+middle+tail context window
      (≤ 4000 chars), formats top-5 candidates, injects a near-tie banner
      if the top two are within 0.005, calls Llama 3.1 8B (temperature 0.1),
      parses strict JSON, validates the chosen series against the candidate
      list (falls back to candidate #1 if the model goes off-list).
      |
      v
[5] Metadata resolution from the candidates already in memory (no extra
      Milvus round trip).
      |
      v
[6] Assemble JSON: data, top_3_candidates, classification, selected_record_metadata,
      out_of_scope, optional warning, optional extraction preview.

```

Typical measured end-to-end latency for a real classification (embedding + search + LLM, no mocks) is roughly **2.3–3.0 seconds** steady-state.

7. Techniques Applied

7.1 Multi-Format Extraction with Fallback Chains

`services/extractionService.js` routes by file extension, with a descending fallback chain for PDFs:

1. **pdf-parse** — fast direct extraction from the content streams.
2. **pdfjs-dist** — triggered on xref / trailer errors; reconstructs text page by page.
3. **OCR** — triggered when extracted text is empty/whitespace; handed to `ocrService.js`.

DOCX uses `mammoth.extractRawText`. XLSX/XLS emits each sheet as a `[Sheet: <name>]` block of CSV. A single shared routine backs both the main upload pipeline (extracts from the Multer buffer) and the gated debug endpoint (downloads from R2 first).

7.2 Selective OCR with a Persistent Worker Pool

`services/ocrService.js` keeps OCR cost bounded:

- **Persistent pool** — 3 Tesseract workers created once at startup and reused, avoiding per-request spin-up.
- **FIFO slot queue** hands a free worker to the next waiter so nothing blocks indefinitely or leaks.
- **Page-level selectivity** — a page with ≥ 10 chars of embedded text skips OCR entirely; only image-only pages are rasterised.
- **High-DPI rendering** at $2.0\times$ scale improves accuracy on small print and faxes.
- **Batched parallelism** — pages processed in batches of `P00L_SIZE` via `Promise.all`.

7.3 Seven-Stage Text Cleaning

`textCleaningService.js` (`cleanExtractedText`) applies deterministic transforms, recording which rules fired:

1. **Line-ending normalisation** — `\r\n\r -> \n`.

2. **Invisible-character removal** — BOM, zero-width spaces.
3. **Boilerplate / citation stripping** — Virginia Code and CFR citations, distribution lines, signature blocks, generic filler stems.
4. **ISO 8601 date normalisation** — multiple date formats -> YYYY-MM-DD.
5. **Page-artifact removal** — page numbers, decorative rules, bare numeric lines.
6. **Abbreviation expansion** — Dept. -> Department, e.g. -> for example, and more.
7. **Deduplication and spacing** — drop repeated lines, collapse blank lines, trim whitespace.

Each stage is individually wrapped: a failure logs a warning and passes text through unchanged rather than aborting.

7.4 Context Windowing for Long Documents

`createDocumentContextWindow()` produces a **head + representative-middle + tail** excerpt for text longer than budget; shorter text passes through untouched. Two call sites tune it differently:

Consumer	maxLength	Head / Middle / Tail
Embedding input	1500	0.40 / 0.20 / 0.40
LLM prompt	4000	0.375 / 0.25 / 0.375

This keeps the beginning, a mid sample, and the end — the regions carrying the most classification signal — within model limits.

7.5 Semantic Vector Embeddings

`embeddingService.js` calls Cloudflare Workers AI (`@cf/baai/bge-base-en-v1.5`) and validates the response is exactly 768 floats. An optional **classification framing** wrapper (`USE_CONTEXT_EMBEDDING`, off by default) prepends a domain primer before embedding. The same service embeds both the reference series (offline) and the live query document into one comparable vector space.

7.6 Vector Similarity Search (ANN)

`milvusService.js` (`searchSimilarRecords`):

- Collection `gs17_records`, metric `COSINE`, index `AUTOINDEX`.
- Query params `{ nprobe: 32 }`, `limit: 8`, `consistency_level: 'Strong'`; retries once without custom params on error.
- Results re-sorted by score and validated for required fields.

`getFullRecord(seriesNumber)` resolves metadata by exact series number, falling back to the local schedule JSON if Milvus is unavailable.

7.7 Stage-1 Reranking — Metadata Keyword Boost

`uploadController.js` counts how many of each candidate's `key_criteria` phrases appear in the cleaned document, then boosts `min(0.030, hitRatio × 0.030)` and re-sorts. Fully schedule-agnostic — the discriminating knowledge lives in the data, not in code.

7.8 Stage-2 Reranking — IDF-Weighted Confusable-Cluster Amplification

`rerankingService.js` (`reRankConfusableClusters`) is the more sophisticated pass:

1. **IDF term weighting** — terms common across many candidates are down-weighted ($1 / \text{documentFrequency}$); a fixed generic-word list is capped at weight **0.10**.
2. **Word-boundary matching** with simple singular/plural inflections.
3. **Exclusion graph & cluster detection (BFS)** — candidates' `exclusions` arrays form a graph; connected components ≥ 2 are "confusable clusters" (e.g. *Resolved* vs. *Unresolved* offenses).
4. **Amplified ceiling** — clustered candidates get up to **+0.120**, others **+0.060**, with sub-linear saturation so broad series aren't unfairly penalised.
5. **Near-tie signal** — top two within **0.005** triggers a disambiguation banner in the LLM prompt.

A 17-case regression suite (`scripts/test_reranking.js`) covers every known GS-17 confusable cluster.

7.9 Similarity Threshold Gating (Out-of-Scope Protection)

`MIN_SIMILARITY_THRESHOLD = 0.65`. Below this after reranking, the controller sets `out_of_scope: true`, **skips the LLM entirely**, and returns the nearest 3 candidates "for reference only" — stopping the model from confidently misfiling something like a payroll spreadsheet as a law-enforcement record.

7.10 Retrieval-Augmented Generation

`classificationService.js (classifyDocument)`:

- *System prompt* defines the role, a 5-step reasoning method (extract facts -> compare against scope notes/key criteria -> respect exclusions -> allow overriding rank #1 when facts point elsewhere -> never extrapolate), and requires the answer to be one of the supplied series numbers.
- *User prompt* — windowed document text + top-5 candidates with schedule, retention, disposition, similarity, scope notes, key criteria, and exclusions; near-tie banner appended when applicable.
- Output is strict JSON: `key_facts_identified`, `selected_series_number`, `reasoning`, `certainty` (HIGH/MEDIUM/LOW).
- Response is stripped of Markdown fences, parsed, and the chosen series is **validated against the candidate list** — falls back to candidate #1 if the model goes off-list.

7.11 Composite Confidence Scoring

Metric	Meaning	Range
<code>similarity_score</code>	Raw cosine similarity from Milvus	0.00 – 1.00
<code>confidence</code>	Final trust rating shown to the user	up to 0.95

`calculateFinalConfidence(similarity, reasoning, explicitCertainty)`:

1. **Certainty detection** — use the model's explicit `certainty` field, or scan the reasoning text for HIGH/MEDIUM/LOW keywords (default MEDIUM).
2. **Boost** — only when certainty is HIGH **and** similarity ≥ 0.65 : `boost = 0.08 + min(0.04, (similarity - 0.65) × 0.2)`, i.e. +0.08 to +0.12.
3. **Final** — `min(0.95, max(baseSimilarity, baseSimilarity + boost))`.

Guarantees: **non-penalising** (final $\geq$ base similarity) and a hard **0.95 ceiling**, reflecting that legal classification always leaves headroom for human sign-off.

7.12 Performance — Concurrency and Round-Trip Elimination

Two redundant network round trips were removed from the original design:

- **Archive and extract now run in parallel** via `Promise.all`, instead of uploading to R2 then re-downloading to extract text.
- **Metadata resolution no longer re-queries Milvus** — the selected series is always one of the candidates already fetched.

Net effect: 2 of roughly 6 external calls per request eliminated; end-to-end latency dropped from ~5 s to ~2.3–3.0 s.

8. Internal REST API Reference

Base: same origin as the served UI. All responses are JSON with a consistent success/failure envelope.

8.1 POST /api/v1/upload

Runs the full pipeline: archive, extract, clean, embed, search, rerank, gate, classify, assemble.

Content-Type	<code>multipart/form-data</code>
Field	<code>document</code> — a single <code>.pdf</code> , <code>.docx</code> , <code>.xlsx</code> , or <code>.xls</code> file
Max size	2 MB
Rate limit	20 requests / 15 minutes per IP

Success — 200 OK (abridged)

```
{
  "success": true,
  "data": { "filename": "...", "originalName": "fairfax-tow-slip.pdf", "size": 45210 },
  "top_3_candidates": [
    { "rank": 1, "series_number": "100812", "schedule_number": "GS-17",
      "series_title": "Abandoned/Impounded Vehicles",
      "retention_period": "3 Years after ... no longer in use",
      "disposition_method": "Non-confidential Destruction",
      "similarity_score": 0.8824 }
  ],
  "classification": {
    "selected_series_number": "100812",
    "ai_reasoning": "The document is a vehicle tow and impound inventory slip ...",
    "confidence": 0.95
  },
  "selected_record_metadata": { "...": "full statutory metadata for 100812" },
  "out_of_scope": false
}
```

Additional / conditional fields

Field	When present
<code>warning</code>	Any soft failure (search, classification, or metadata)
<code>extraction</code>	Only when <code>ENABLE_EXTRACTION_PREVIEW=true</code>
<code>classification: null</code> + <code>out_of_scope: true</code>	Top similarity below 0.65

Field	When present
<code>classification: null + warning</code>	Search returned nothing, or the LLM call failed

Errors

Status	Cause
400	No file, disallowed MIME type, or file over 2 MB
429	Rate limit exceeded
500	Unexpected error (stack trace only in development)

8.2 `POST /api/v1/extract` (development only)

Extracts and cleans text for an existing R2 object key. **Disabled unless `ENABLE_EXTRACTION_PREVIEW=true`** (404 otherwise); no ownership check; not used by the bundled UI; must stay off in production.

Content-Type	<code>application/json</code>
Body	<code>{ "r2Key": "1771586390000-sample-report.pdf" }</code>
Rate limit	30 requests / 15 minutes per IP

Success — `{ "success": true, "fileType": "pdf", "text": "..." }`. **Errors** — 400 (missing/unsupported), 404 (disabled or key not found), 422 (corrupt file), 429, 500.

8.3 `GET /api/health`

<code>{ "success": true, "project": "AI-Based Document Classification System", "status": "Running" }</code>

8.4 `GET /`

Serves `views/index.html`. The `__APP_CONFIG__` placeholder is replaced at serve time with `{ "enableExtractionPreview": <bool> }`.

9. External APIs and Services

9.1 Cloudflare Workers AI — Embeddings

Endpoint	<code>POST .../ai/run/@cf/baai/bge-base-en-v1.5</code>
Auth	<code>Authorization: Bearer {CLOUDFLARE_API_TOKEN}</code>

Request / Response	<pre>{ "text": "... " } -> { result: { data: [...768 floats...] } }</pre>
Validation	Array length must equal 768, else the call throws

9.2 Cloudflare Workers AI — LLM

Endpoint	<code>POST</code> <code>.../ai/run/@cf/meta/llama-3.1-8b-instruct</code>
Request	<pre>{ messages: [system, user], temperature: 0.1 }</pre>
Response text	<code>result.response</code> (or <code>result.choices[0].message.content</code>)
Post-processing	Strip fences -> extract outermost <code>{...}</code> -> parse -> validate series against candidates

9.3 Zilliz Cloud / Milvus

SDK	<code>@zilliz/milvus2-sdk-node ^3.0.4</code>
Connection	<pre>{ address: MILVUS_ADDRESS, token: MILVUS_TOKEN }, verified via getVersion()</pre>
Collection	<code>gs17_records</code> (holds all schedules despite the name)
Search	<pre>data: [queryVector], limit: 8, params: { nprobe: 32 }, consistency_level: 'Strong'</pre>

9.4 Cloudflare R2

SDK	<code>@aws-sdk/client-s3 ^3.1081.0</code> , region: 'auto', custom endpoint
Object key	<code>\${Date.now()}-\${sanitizedBaseName}\${ext}</code>
Operations	<code>PutObjectCommand</code> (archive on upload), <code>GetObjectCommand</code> (dev extract endpoint only)

10. Data Model

10.1 Schedule JSON Record

Each object in `backend/data/schedules/<schedule>.json` (`records[]`):

Field	Notes
<code>schedule_number</code> / <code>schedule_title</code>	"GS-17" etc. and its human-readable name
<code>series_number</code> / <code>series_title</code>	Unique id (e.g. "100812") and official name
<code>series_description</code>	Statutory description
<code>series_retention_period</code> / <code>series_disposition_method</code>	e.g. "100 Years after birth" / "Confidential Destruction"
<code>text_to_embed</code>	Rich semantic summary — the text that is vectorised
<code>scope_notes</code>	Prose: what the series covers / does not cover
<code>key_criteria</code>	string[] — discriminating keywords/phrases
<code>exclusions</code>	string[] — series numbers this one is commonly confused with

`scripts/validate_schedules.js` asserts every record has all eleven keys populated and correct types.

10.2 Milvus Collection Schema (`gs17_records`)

Mirrors the statutory fields above as `VarChar` columns plus an `id` primary key and an `embedding FloatVector(768)` field (`AUTOINDEX`, `COSINE`). The enrichment fields (`scope_notes`, `key_criteria`, `exclusions`) are **not** stored in Milvus — they're read from local JSON at request time and merged onto each candidate by `series_number`, avoiding a schema migration.

10.3 Dataset Coverage

Schedule	Series	Notes
GS-17	88	Fully embedded and enriched; benchmarked
GS-02	20	Ingested and ambiguity-audited
GS-14	11	Ingested and ambiguity-audited

11. Configuration

All configuration is environment-driven (`config/app.config.js`). Secrets live in `backend/.env` (git-ignored); `.env.example` is the template.

Variable	Default	Purpose
<code>PORT</code>	<code>3000</code>	HTTP port (Render supplies its own)

Variable	Default	Purpose
<code>NODE_ENV</code>	<code>development</code>	<code>production</code> suppresses stack traces
<code>ENABLE_EXTRACTION_PREVIEW</code>	<code>false</code>	Enables <code>/api/v1/extract</code> + UI preview panel
<code>USE_CONTEXT_EMBEDDING</code>	<code>false</code>	Prepends classification framing primer before embedding
<code>CORS_ORIGIN</code>	<i>(unset)</i>	Allowlist; unset = block all cross-origin requests
<code>R2_ACCOUNT_ID</code> / <code>R2_ACCESS_KEY_ID</code> / <code>R2_SECRET_ACCESS_KEY</code> / <code>R2_BUCKET_NAME</code> / <code>R2_ENDPOINT</code>	—	Cloudflare R2 credentials and target bucket
<code>MILVUS_ADDRESS</code> / <code>MILVUS_TOKEN</code>	—	Zilliz cluster URL and API token
<code>CLOUDFLARE_ACCOUNT_ID</code> / <code>CLOUDFLARE_API_TOKEN</code>	—	For Workers AI

Key application constants:

Constant	Value
<code>MAX_FILE_SIZE</code>	2 MB
Upload / Extract rate limit	20 / 15 min · 30 / 15 min
<code>SEARCH_CANDIDATE_LIMIT</code> / <code>CLASSIFICATION_CANDIDATE_LIMIT</code>	8 / 5
<code>MIN_SIMILARITY_THRESHOLD</code>	0.65
Stage-1 boost ceiling	+0.030
<code>STANDARD_MAX_BOOST</code> / <code>CLUSTER_MAX_BOOST</code>	0.060 / 0.120
<code>NEAR_TIE_EPSILON</code>	0.005
Confidence ceiling / LLM temperature	0.95 / 0.1
Embedding dimension	768
OCR pool size / render scale / min text	3 / 2.0× / 10 chars

12. Security

Measure	Implementation
CORS default-deny	<code>cors({ origin: false })</code> unless <code>CORS_ORIGIN</code> is set; the bundled UI is same-origin.
Rate limiting	Per-IP on both cost-sensitive endpoints, protecting paid downstream APIs (429 on breach).
Extract endpoint gated	404 unless <code>ENABLE_EXTRACTION_PREVIEW=true</code> ; no ownership check; dev-only.
Upload validation	MIME whitelist in Multer's <code>fileFilter</code> ; 2 MB hard limit.
In-memory buffers	Uploaded documents never written to local disk; archived to R2 only.
Filename sanitisation	R2 object keys strip non-alphanumeric characters.
No secrets in code or logs	All credentials via environment variables; <code>.env</code> , <code>uploads/</code> , logs are git-ignored.
No stack traces to users	Included only when <code>NODE_ENV === 'development'</code> .
XSS-safe rendering	Frontend escapes all server strings via <code>escapeHTML()</code> before DOM insertion.
Dependency CVE patching	<code>pdfjs-dist</code> upgraded 6.1.200 -> 6.3.289 (arbitrary-JS-execution CVE via untrusted PDFs), plus <code>qs</code> , <code>@xmldom/xmldom</code> , <code>brace-expansion</code> . <code>xlsx</code> retained despite an unpatched advisory — used only to read cell text into a string, never as a config/prototype source.

13. Error Handling Strategy

Principle: user input must never crash the server, and every failure returns consistent JSON.

Failure	Behaviour
Disallowed MIME / oversize / missing file	HTTP 400 — pipeline never starts
Extraction fails (all fallbacks exhausted)	Soft — <code>extraction.error</code> set; search skipped if no text
R2 archive fails	Hard — whole request fails
Milvus search fails / LLM call fails	Soft — <code>warning</code> added, <code>classification</code> omitted / null
LLM picks a non-candidate series	Logged, falls back to candidate #1
Metadata missing from candidates	Defensive live <code>getFullRecord()</code> ; else <code>warning</code>

Failure	Behaviour
Similarity below 0.65	<code>out_of_scope: true</code> , LLM skipped by design
Any uncaught exception	HTTP 500, JSON envelope, stack only in development

14. Diagnostic and Maintenance Tooling

CLI scripts in `backend/scripts/`, exposed as npm scripts.

Script	Purpose
<code>enrich-and-reembed-all.js</code>	Rewrites and re-embeds <code>text_to_embed</code> for all 88 GS-17 series
<code>ingest-new-schedules.js</code>	Idempotently embeds and inserts GS-02 / GS-14
<code>reembed-series.js</code> / <code>reenrich-series.js</code>	Targeted re-embed / re-enrich of specific series
<code>diagnose-milvus.js</code> (<code>npm run diagnose</code>)	Env, connection, collection health, zero-vector detection, live search sample
<code>benchmark-all-88.js</code> (<code>npm run benchmark</code>)	Top-1 / Top-3 / Top-5 retrieval accuracy across all 88 GS-17 series
<code>audit-cross-schedule.js</code> (<code>npm run test:audit</code>)	Synthetic per-series test docs for GS-02 / GS-14; flags series needing enrichment
<code>validate_schedules.js</code> (<code>npm run test:schedules</code>)	Schema / completeness validation of all schedule JSON
<code>test_reranking.js</code> (<code>npm run test:reranking</code>)	17-case regression suite for the confusable-cluster reranker
<code>test-area-a-reasoning.js</code> (<code>npm run test:reasoning</code>)	LLM reasoning / prompt verification against curated cases

Aggregate: `npm test` (reranking + schedules), `npm run test:all` (adds reasoning).

15. Frontend

Single page, zero frameworks — `views/index.html` with `public/css/style.css`, `public/js/theme.js`, `public/js/upload.js`.

Aspect	Detail
Design	Glassmorphism — translucent panels, backdrop blur, Inter / system fonts

Aspect	Detail
Theme	Dark / light toggle, persisted in <code>localStorage</code> ; applied before first paint to avoid a flash
Upload UX	Click or drag-and-drop; client-side extension check; size formatting
Request	<code>fetch('/api/v1/upload', { method: 'POST', body: FormData })</code> with a loading spinner
Results rendering	Classification header + confidence badge, metadata grid, AI reasoning card, top-3 candidates with similarity %
Out-of-scope	Dedicated warning card; candidates shown "for reference only"
Safety	Every server string passes through <code>escapeHTML()</code> before DOM insertion

16. Setup and Local Development

16.1 Prerequisites

- Node.js $\geq$ **22.13**.
- Accounts / credentials for Cloudflare (Workers AI + R2) and Zilliz Cloud.

16.2 Install and Configure

```
cd backend
npm install
cp .env.example .env
# then edit .env and fill in every value (see §11)
```

16.3 One-Time Data Load

```
node scripts/enrich-and-reembed-all.js # GS-17 (88 series)
node scripts/ingest-new-schedules.js   # GS-02 + GS-14
node scripts/diagnose-milvus.js        # verify: connection, counts, live search
```

16.4 Run

```
npm run dev # nodemon, auto-restart
# or
npm start  # plain node
```

Open <http://localhost:3000>.

16.5 Test

```
npm test          # reranking regression + schedule validation
npm run benchmark # retrieval accuracy across all 88 GS-17 series
```

17. Deployment

Target platform: **Render.com** free web service (see [how-to-live.md](#) for the full walkthrough).

Setting	Value
Root directory	<code>backend</code>
Build / Start command	<code>npm install / npm start</code>
Region	Oregon (US West) — matches the Zilliz cluster to minimise latency
Environment	All §11 variables except <code>PORT</code> ; <code>NODE_ENV=production</code> , <code>ENABLE_EXTRACTION_PREVIEW=false</code>

The free tier sleeps after 15 minutes idle; the first request after sleep takes ~30 seconds to wake.

18. Future Enhancements

The system is feature-complete for the internship scope; the architecture allows each of these without disturbing existing modules:

- **Result persistence and audit trail** — store classifications with reviewer confirm/override.
- **Authentication and per-agency workspaces.**
- **Batch processing** via an async job queue.
- **Additional LVA schedules** — the engine is already schedule-agnostic; mostly a data task.
- **Enrichment fields in Milvus** — retire the request-time local-JSON merge.
- **Feedback loop** — feed reviewer corrections back into `key_criteria` / `text_to_embed`.
- **Formal evaluation harness** — labelled gold set with precision/recall per schedule.

Appendix A: Glossary

Term	Definition
ANN	Approximate Nearest Neighbour — fast vector search trading exactness for speed
BGE	The <code>baai/bge-base-en-v1.5</code> embedding model family

Term	Definition
Confusable cluster	Candidate series that mutually list each other in <code>exclusions</code>
Cosine similarity	Similarity by vector angle; 1 = same direction
Disposition method	Legally mandated end state of a record
Embedding	Fixed-length numeric vector representing text meaning
GS-17 / GS-02 / GS-14	LVA General Schedules covered by this project
IDF	Inverse Document Frequency — down-weights terms common across candidates
RAG	Retrieval-Augmented Generation
Retention period	How long a record must legally be kept
Series	One record category within a schedule
Similarity gate	The 0.65 threshold below which classification is refused as out-of-scope
<code>text_to_embed</code>	Curated per-series summary string that gets vectorised

Appendix B: File Reference Map

Full directory tree with descriptions is in §4.3; key files by layer:

Path	Responsibility
<code>server.js</code>	Express app, middleware order, listen
<code>config/</code>	Environment -> config object, upload/rate-limit constants
<code>middleware/</code>	Multer memory storage, per-IP limiters, 404 + global error handler
<code>routes/</code>	Endpoint definitions: UI, health, upload, gated extract
<code>controllers/uploadController.js</code>	Pipeline orchestration, stage-1 rerank, response assembly
<code>services/extractionService.js</code> + <code>ocrService.js</code>	Format routing, fallback chain, Tesseract worker pool
<code>services/textCleaningService.js</code>	7-stage cleaning, context-window builder

Path	Responsibility
<code>services/embeddingService.js + milvusService.js</code>	768-dim embeddings, connection, search, lookup
<code>services/rerankingService.js</code>	IDF weighting, exclusion graph, cluster BFS
<code>services/classificationService.js</code>	RAG prompt, LLM call, JSON validation, confidence scoring
<code>services/uploadService.js + r2Service.js</code>	R2 upload, key generation, put/get
<code>data/schedules/*.json</code>	Enriched series datasets (GS-17, GS-02, GS-14)
<code>views/, public/</code>	UI markup, design system, theme toggle, upload flow
<code>scripts/*.js</code>	Diagnostics, benchmarks, ingestion, tests (§14)

End of Technical Documentation — Version 1.0